AIRO TECHNICAL GOVERNANCE

# Agentic Skills & Capabilities Directory

Architecture, Roles, Responsibilities & Upstream Git Repositories

**Project:** AIRO Essentials & ONE
**Stack:** Next.js 14, Tailwind, Firestore
**Scope:** Core Design & UI Intelligence

## 1. Design Intelligence & Visual Systems

### ui-ux-pro-max

DESIGN INTELLIGENCE

`Source: https://github.com/nextlevelbuilder/ui-ux-pro-max-skill`

**Primary Role:** Comprehensive design reasoning system providing deep UI/UX rules, accessible typography, color psychology, and responsive layouts.

- **Design Reasoning:** Employs 192 reasoning profiles, 79 searchable styles, and 119 UX guidelines tailored to product domains.
- **Color & Typography:** Selects curated product palettes, contrast ratios, and tailored Google Font pairings (Outfit, Inter).
- **Ergonomics & Mobile:** Enforces minimum 48px tap targets, thumb-friendly actions, and smooth responsive viewport scaling.
- **Stack Adaptation:** Translates design concepts into production Next.js, React, and CSS implementations.

> **WHEN TO TRIGGER**
> Designing, building, or auditing pages, modal sheets, checkout funnels, navigation bars, or responsive cards.

### frontend-design

VISUAL IDENTITY

`Source: https://github.com/anthropics/claude-code (plugins/frontend-design)`

**Primary Role:** Creative art direction ensuring distinctive visual identity and preventing templated "AI-generated" looks.

- **Editorial Art Direction:** Enforces authentic editorial typography, tactile warm cream backgrounds, and rich dark emerald accents.
- **Anti-Generic Guardrails:** Prohibits cliché single-word bolding, gratuitous numbered tags, and over-decorated layouts.
- **Content & Cadence:** Optimizes line lengths (<80 chars), clear hierarchy, and intentional breathing room.

> **WHEN TO TRIGGER**
> Redesigning hero sections, membership showcase views, doctor portals, and establishing brand voice.

### apple-design & emil-design-eng

TACTILE POLISH

**Primary Role:** Premium interaction and craftsmanship standards based on Apple Human Interface Guidelines and Emil Kowalski's micro-interaction principles.

- **Depth & Materials:** Subtle 1px translucent borders, glassmorphic backdrop blurs, and layered elevations.
- **Tactile Press Feedback:** Micro-spring animations on buttons, active click scaling (`active:scale-95`), and tactile affordances.
- **Invisible Details:** Proportional tracking, optical text sizing, smooth pill badges, and refined scrollbar treatments.

> **WHEN TO TRIGGER**
> Polishing buttons, dropdowns, card borders, interactive states, and dialog modals.

COMPONENT ENGINEERING & MOTION

# Components, Catalogs & Animations

Pre-Built Catalogs, Motion Choreography & Notification Architecture

**Section:** 2 & 3 • UI Components & Motion
**Tools:** 21st.dev CLI, Framer Motion, Sonner

## 2. 21st.dev Component Catalog & Prototyping

### 21st-dev Suite (ui-build, cli-use, ai, explore, review)

COMPONENT SUITE

`Source: https://github.com/21st-dev (@21st-dev/cli)`

**Primary Role:** Terminal-to-code integration with the 21st.dev ecosystem for rapid React/shadcn component discovery, generation, and synchronization.

- **`21st-cli-use`:** Search, inspect, and install production components and theme palettes directly from 21st.dev catalog.
- **`21st-ui-build`:** Construct production pages using the project's design tokens and grounded component inspiration.
- **`21st-ui-explore`:** Generate and compare multiple meaningfully distinct UI directions for rapid stakeholder feedback.
- **`21st-ui-review`:** Design QA audit against responsive behavior, accessibility guidelines, and contrast standards.
- **`21st-design-sync` & `21st-registry`:** Sync local CSS variables into community themes and publish reusable packages.

> **WHEN TO TRIGGER**
> Scaffolding new screens, searching for modern shadcn UI blocks, or exploring comparative UI directions.

## 3. Motion, Transitions & Micro-Interactions

### animate, find-animation-opportunities & improve-animations

MOTION ENGINEERING

**Primary Role:** Motion choreography advisor ensuring deliberate, purposeful physics-based transitions without performance degradation.

- **Purpose-Driven Motion:** Chooses exact transition timing, cubic-bezier curves, spring parameters, and exit behaviors.
- **Performance First:** Limits animations strictly to composite properties (`transform`, `opacity`) to eliminate layout thrashing.
- **Opportunity Auditing:** Identifies static UI moments (e.g., leaflet page turn, card flip, accordion expand) that benefit from motion.
- **Accessibility:** Respects `prefers-reduced-motion` across all animated surfaces.

> **WHEN TO TRIGGER**
> Building carousels, page transition flippers, interactive modals, tabs, or smooth collapsing accordions.

### ask-sonner

NOTIFICATION SYSTEM

**Primary Role:** Production implementation guide for Sonner, the high-performance React toast library.

- **Toast Lifecycle:** Dispatches success, error, loading, and promise-based transactional toasts.
- **Theme Integration:** Styles toasts with brand emerald/dark-charcoal colors and ensures z-index layering above dialogs.

> **WHEN TO TRIGGER**
> Confirming actions (plan selection, feedback submission, PDF download triggers, form saves).

CODE GOVERNANCE & VERIFICATION

# Code Quality, Backend & Verification

Radical Simplicity, Cross-Domain SSO & In-Browser Automated Proof

**Section:** 4 & 5 • Code Quality & Verification
**Engines:** Ponytail, Firebase, Reticle

## 4. Senior Engineering Minimalist Standards (Ponytail)

### ponytail & ponytail-review (audit, debt, gain)

CODE GOVERNANCE

`Source: https://github.com/dietrichgebert/ponytail`

**Primary Role:** Senior developer philosophy focused on radical simplicity, YAGNI (You Aren't Gonna Need It), and zero unnecessary code.

- **Standard Library Priority:** Uses native browser/Node APIs before adding heavy third-party npm packages.
- **Anti-Bloat Review:** Flags speculative abstractions, dead code, redundant wrappers, and over-engineered boilerplate.
- **Debt Tracking (`ponytail-debt`):** Explicitly logs deliberate shortcuts so they are tracked systematically.
- **Speed & Maintenance:** Keeps codebase size minimal, accelerating compilation and reducing bug surface area.

> **WHEN TO TRIGGER**
> Reviewing PRs, refactoring features, choosing architecture patterns, or simplifying complex components.

## 5. Backend, Authentication & Verification Layer

### firebase-firestore & firebase-auth-basics

DATA & AUTH

**Primary Role:** Manages the unified multi-domain cloud database, customer records, and Single Sign-On (SSO) infrastructure.

- **Cross-Domain SSO:** Powers seamless customer authentication across `airoessentials.com` and `airohealthhub.com`.
- **Unified Firestore Schema:** Manages real-time data for memberships, clinical orders, Praana vital scans, and appointments.
- **Security Rules:** Enforces granular read/write authorization rules protecting sensitive patient health records (EHR).

> **WHEN TO TRIGGER**
> Querying member statuses, storing consultation records, or authenticating users.

### reticle (In-App Verification)

E2E VERIFICATION

`Source: https://github.com/reticlehq/reticle (@reticlehq/server)`

**Primary Role:** In-browser verification layer executing automated, assertion-backed journeys before declaring tasks complete.

- **Flow Proof:** Drives flows using `reticle_act_and_wait` to verify consequences rather than guessing.
- **Telemetry Inspection:** Captures state snapshots, network calls, console logs, and DOM changes during flow execution.
- **Regression Guard:** Evaluates affected user paths against recorded benchmarks to prevent regressions.

> **WHEN TO TRIGGER**
> Every time user-visible components, checkout flows, or interactive forms are modified.